Smart City Traffic Intelligence: From Data Analytics to AI-Powered Mobility

Final Capstone Report | Prepared by: **Kyi Htoo Aung** | NUS SoC AI, ML and Data Science Programme | Dataset: Metro Interstate Traffic Volume (westbound I-94, Minneapolis-St Paul, 2012-2018)

1. Executive summary

This project turns 48,204 raw hourly traffic records into an end-to-end traffic intelligence solution for a Smart City Mobility Analytics Team. Part 1 explored the data with SQL, statistics, probability and Power BI. Part 2 built a logged, reproducible Python pipeline that cleans the data, engineers features and serves a command-line tool. Part 3 added machine learning, deep learning, explainability, MLOps, a travel-timing recommender and a responsible-AI assessment. Each part reuses the outputs of the one before it.

- **Congestion on this corridor is a weekday commuting problem.** Volume exceeds 5,500 vehicles per hour mainly on weekdays at 07:00-08:00 and 15:00-17:00 (94-96% of Tuesday-Thursday 16:00 hours). Weekends are almost never congested.
- **Time of day matters far more than weather.** Temperature correlates weakly with volume (r = 0.13). Clear weather appears to reduce congestion (odds ratio 0.735), but the effect vanishes once hour of day is controlled (1.006). Clustering, association rules and SHAP in Part 3 all confirm this independently.
- **Traffic is highly predictable.** From calendar and weather alone, gradient boosting explains 96% of the variation on unseen 2017-2018 data (MAE 233 vehicles per hour). Adding the previous 24 hours, an LSTM predicts the next hour with MAE 180, three times better than naive baselines.
- **The solution is operational, not just analytical:** every model version is tracked and registered in MLflow, the champions are served through a FastAPI mock-up, a recommender turns predictions into plain-language travel advice, and monthly monitoring reports PASS or ALERT (it caught a real April 2018 snowstorm).
- **Limits are stated plainly.** The "accident risk" target is a documented proxy, not accident data; errors are larger in severe weather and on holidays; the data come from one sensor with a 10-month gap.

2. Data and overall approach

The dataset has nine columns: traffic volume (the hourly vehicle count), temperature in Kelvin, rain and snow in the last hour, cloud cover, two weather labels, a holiday label and a timestamp. It runs from 2 October 2012 to 30 September 2018. The work followed one data flow:

Stage	Input	What was done	Output reused later
Part 1: Data analytics	Raw CSV	SQLite queries, descriptive statistics, correlation, probability, Power BI dashboard	Data-quality findings, peak hours, the 5,500 congestion threshold
Part 2: Python pipeline	Raw CSV + Part 1 findings	Validation, 12 logged cleaning steps, feature engineering, 6 figures, CLI app	Clean feature table (40,575 hours x 54 columns)
Part 3: ML and AI	Part 2 feature table	Supervised, unsupervised and deep learning, SHAP/LIME, MLflow, recommender, API, monitoring	Models, registry, recommendations, monitoring status

3. Part 1: Data analytics

3.1 SQL-based analysis

The CSV was loaded into SQLite and verified (48,204 rows, no nulls, but ten temperatures of 0 K, one rainfall reading of 9,831 mm, 17 duplicate rows and 7,629 repeated timestamps). Raw yearly totals swing wildly (+243% in 2013, -44% in 2014, +108% in 2016) but these swings track **data coverage, not demand**: there is no data from August 2014 to June 2015 and coverage ranges from 24% to 99.5% of hours per year. Counting each hour once, average volume was flat at about 3,190-3,310 vehicles per hour from 2012 to 2016 and rose 5.7% to 3,377 in 2017. Holiday analysis showed New Year's Day 2017 was 4.8 °C milder than 2016 and Labor Day cooled each year, yet every holiday carried 33-48% less traffic than comparable days regardless of temperature: the calendar, not the temperature, drives holiday traffic.

3.2 Statistics, correlation and probability

Hourly volume has mean 3,260, median 3,380, standard deviation 1,987 and range 7,280. The distribution is bimodal (quiet nights, busy days), so a single average describes few real hours. Temperature and volume are weakly positively correlated (Pearson r = 0.13, under 2% of variance); the link is confounded by time of day and season, so it is not causal. With congestion defined as volume above 5,500: P(Congestion) = 0.147, P(Clear) = 0.278, P(Congestion and Clear) = 0.037, P(Clear | Congestion) = 0.248 and P(High temperature | Congestion) = 0.263. The joint probability is below the product of the marginals (0.041), so weather and congestion are not strictly independent (chi-square $p < 0.001$), but the dependence is weak. The odds of congestion in clear versus cloudy weather are 0.735, falling to 1.006 after adjusting for hour, because clear skies are more common at night.

3.3 Power BI dashboard

Power Query fixed mis-detected data types (rain and snow as decimals), removed duplicates and added Hour, Temperature (C) and Traffic Category. The interactive dashboard shows daily trends for 2015-2017, the 2017 hourly profile (lowest at 03:00, peaks at 07:00 and 16:00), average traffic by weather (Clouds highest at 3,618; Squall lowest at 2,062 but with only four records; Fog the reliable low at 2,704) and a temperature scatter plot, with KPI cards (40,575 hours, 3,260 average volume, 8.1 °C) and slicers for hour, weather and traffic category.

4. Part 2: Reproducible Python pipeline

pipeline.py is a single entry point that loads the CSV inside specific exception handlers, validates the schema before any processing, and runs twelve cleaning steps that each write their own log message. It standardised 1,730 inconsistent labels ("Sky is Clear" vs "sky is clear"), removed 17 exact duplicates, collapsed 7,612 repeated-hour rows to one row per hour (keeping the most severe weather report), replaced impossible temperatures and rainfall with the **median of the same month** in an explicit loop (the 0 K readings fell in January-February 2014, where the monthly median is about -13 °C against a global median of +8 °C) and flagged 32 sensor-fault hours. The result is 40,575 clean hourly rows.

feature_engineering.py added 40 features: hour, day of week, weekend, holiday (spread from the midnight-only label to the whole day), peak-hour flag, sine/cosine encodings of hour, weekday and month, one-hot weather, precipitation, low-visibility and severe-weather indicators, scaled numerics, and a quartile-based congestion category (thresholds 1,254.5, 3,431 and 4,953). **visualizations.py** saved six figures. A command-line application answers six kinds of question (summary, a specific hour, peak periods, weekday vs weekend, recommended hours, weather) and rejects invalid input with a clear error. Logging follows the brief: a named logger per module, handlers only in entry points, INFO for milestones, WARNING for every changed row with count and reason, ERROR with traceback on failure, DEBUG only in debug mode, and no print for status.

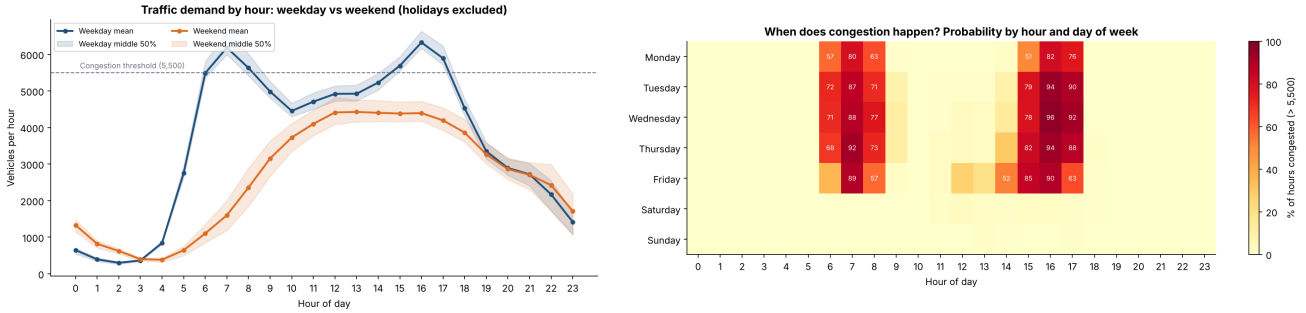


Figure 1. Part 2: weekday commuter peaks exceed the congestion threshold (left); congestion is confined to weekday peak hours (right).

5. Part 3: Machine learning and AI

Accident data: a documented proxy was used. No accident dataset was sourced. Following the brief, *high_risk* = High or Severe congestion (quartile-based) occurring together with severe weather (Thunderstorm, Squall, Snow) or low visibility (Mist, Fog, Haze, Smoke). It marks 7.7% of hours. It demonstrates the classification workflow only and must not be interpreted as a prediction of actual accidents.

5.1 Task 1: Supervised models

Both models use one common set of 31 features: time features, cyclical encodings of hour, day of week and month, the holiday flag and weather encodings. Traffic volume itself is never an input. Models were trained on 2012-2016 (25,297 hours) and tested on unseen 2017-2018 data (15,246 hours); a chronological split is used because a random split would let the model see the future. Three algorithms were trained per problem: a linear baseline and two tree ensembles.

Regression (traffic volume)	MAE	R^2	RMSE	Train time	Size
Ridge	549	0.872	708	0.1 s	0.0 MB
RandomForestRegressor	228	0.964	374	4.7 s	4.4 MB
HistGradientBoostingRegressor (champion)	233	0.963	379	2.5 s	0.9 MB

Classification (proxy risk)	Accuracy	Precision	Recall	F1	ROC AUC	ROC AUC, bad-weather hours only
LogisticRegression	0.979	0.817	0.987	0.894	0.998	0.981
RandomForestClassifier	0.989	0.899	0.992	0.943	0.998	0.986
HistGradientBoostingClassifier (champion)	0.991	0.924	0.976	0.949	0.997	0.985

Regression. The tree ensembles cut the baseline's error by more than half (MAE 549 to about 230) because traffic depends on interactions (hour x weekday x holiday) that a linear model cannot express. The two ensembles are practically tied, so the smaller gradient-boosting model was made champion (within 0.005 R^2 at one fifth of the size). **Classification.** Scores are very high, and this needs careful reading: the label is built from weather, which is also an input, so every clear-weather hour can be ruled out directly and the overall scores are inflated. Every positive hour has severe or low-visibility weather and no model raises a false alarm elsewhere, so F1 restricted to those hours equals the overall F1 by construction and adds no information. The informative check ranks hours only within severe or low-visibility weather, where the model must genuinely predict congestion: the champion's ROC AUC falls from 0.997 to 0.985. This is skill at predicting congestion in bad weather, not accidents. Because only 7.7% of hours are positive, class weighting was used and precision, recall and F1 are more informative than accuracy. Logistic regression has the lowest precision (0.817: more false alarms); gradient boosting gives the best balance.

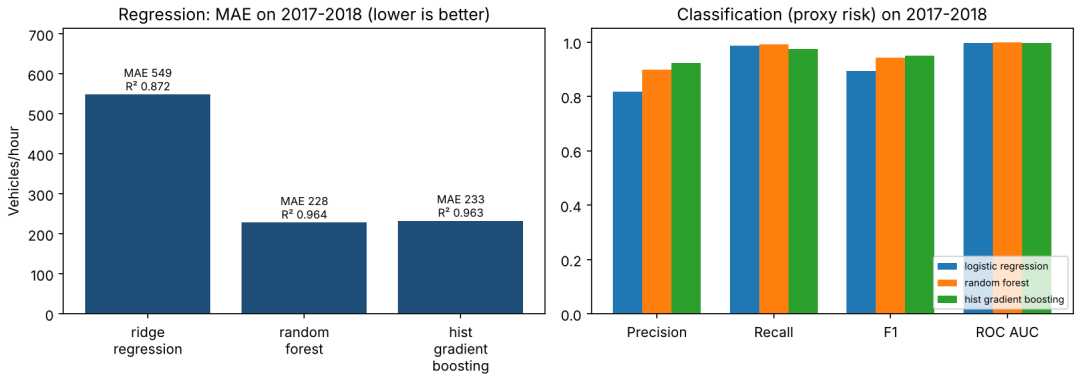


Figure 2. Model comparison on the 2017-2018 test period.

5.2 Task 2: Unsupervised learning

K-means clustering. Hours were clustered on hour of day (cyclical), weekday/weekend, weather severity and traffic volume, all standardised. Silhouette scores were computed for k = 2 to 12. The score keeps rising to k = 11 because the yes/no weekend flag and stepped severity scale allow ever finer splits, so k = 8 was chosen as the best score (0.371) within an interpretable range of 3-8 regimes. The clusters are:

Cluster (plain-language name)	Share of hours	Mean volume	% congested	% weekend	% bad weather
Quiet weekday 00-05h	14%	880	0%	0%	0%
Quiet weekend all day	12%	1,180	0%	100%	11%
Moderate weekday all day in bad weather	10%	1,556	0%	20%	97%
Moderate weekday 19-23h	12%	2,548	0%	0%	0%
Busy weekend 10-20h	13%	3,897	1%	100%	7%
Peak-congestion weekday 07-17h in bad weather	10%	4,942	28%	16%	98%
Peak-congestion weekday 06-12h	16%	5,190	35%	0%	0%
Peak-congestion weekday 13-18h	14%	5,440	48%	0%	0%

The clusters separate naturally into quiet nights, weekend patterns and weekday peaks. Two findings stand out. The afternoon weekday peak (13-18h) is the most congested regime (48%), ahead of the morning (35%). And the **bad-weather peak cluster is less congested (28%) than the fair-weather peaks** at a similar time of day: bad weather slightly suppresses demand rather than creating congestion, consistent with Part 1.

Association rule mining. Each hour became a transaction of five items: time band, day type, weather group, temperature band and congestion level. Apriori (minimum support 1%, confidence 50%) was run and rules predicting a

congestion level were kept. Longer rules that added under 2% lift over a simpler rule were pruned, leaving 42 rules. Because the congestion levels are quartiles, each has a 25% base rate, so a lift of 3.7 means the outcome is 3.7 times more likely than usual. The strongest rules:

- When it is Weekday and Cold and AM peak (06-09), congestion is Severe in 92% of hours, 3.7 times more often than the 25% seen overall. *(lift 3.68, support 2.3%)*
- When it is Weekend and Warm and PM peak (15-18), congestion is High in 92% of hours, 3.7 times more often than the 25% seen overall. *(lift 3.67, support 1.6%)*
- When it is Weekday and Mild and AM peak (06-09), congestion is Severe in 90% of hours, 3.6 times more often than the 25% seen overall. *(lift 3.62, support 3.5%)*
- When it is Weekday and AM peak (06-09) and Low visibility, congestion is Severe in 90% of hours, 3.6 times more often than the 25% seen overall. *(lift 3.62, support 1.5%)*
- When it is Weekend and Freezing and Night (23-05), congestion is Low in 88% of hours, 3.5 times more often than the 25% seen overall. *(lift 3.54, support 2.4%)*

In plain terms: a weekday morning peak means severe congestion about nine times in ten, whatever the weather (cold, mild, rain or low visibility all appear with nearly the same lift). Weather barely changes the rule; the time and day type create it. The low-visibility rule is exactly the combination the proxy risk label flags.

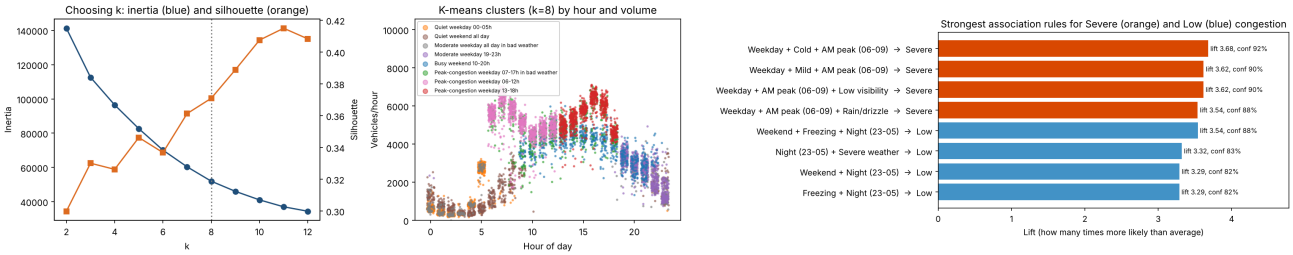


Figure 3. Choosing k and the resulting clusters (left); strongest rules for Severe and Low congestion (right).

5.3 Task 3: Deep learning with explainability

Model. An LSTM was chosen because traffic is sequential. It predicts the next hour's volume from the previous 24 hours (volume, time encodings, weekend, holiday, temperature, weather severity, precipitation) plus the known context of the target hour. Architecture: LSTM(48), dropout, concatenation with 10 context features, Dense(32), Dense(1); 13,249 parameters; Adam optimiser, mean-squared-error loss, up to 80 epochs with early stopping; scaling fitted on training years only. Only windows of 24 consecutive recorded hours were used (14,218 training and 14,483 test windows), so sensor gaps are never bridged.

Next-hour prediction, test 2017-2018	MAE	R^2
LSTM	180	0.981
Naive: same as previous hour	592	0.826
Naive: same hour yesterday	561	0.732
Tree model with lag features (SHAP)	154	0.986

The LSTM is about three times more accurate than the naive baselines (MAE 180 against 561-592). A gradient-boosting model given hand-made lag features is slightly better still (MAE 154), a common result for modest-sized tabular time series, and it trains about 15 times faster.

Explainability and the choice of model to explain. SHAP is difficult to apply reliably to an LSTM, so, as the brief permits, SHAP (TreeExplainer) was applied to the comparable tree model trained on the same next-hour problem. The most influential features by mean absolute SHAP value are lag_1h (1,209 vehicles), hour_cos (496) and change_last_hour (259); weather contributes almost nothing (temperature 11, weather severity 5). To check that this substitution is fair, permutation importance was computed on the LSTM itself: shuffling the target hour's time encoding or the recent volume sequence raises its error by about 1,000-1,100 vehicles, while weather inputs matter little. Both models rely on the same signals. For a single weekday 07:00 forecast of 6,613 vehicles, the SHAP waterfall shows the last hour's volume (+1,429), the rise over the last hour (+955) and the time of day (+612) lifting the prediction above the 3,260 average. **LIME** was applied to the proxy-risk classifier for two foggy hours: at 09:00 on a Tuesday (probability 1.00) high weather severity and the daytime hour push towards high risk; at midnight (probability 0.00) the same fog is outweighed by the night-time hour.

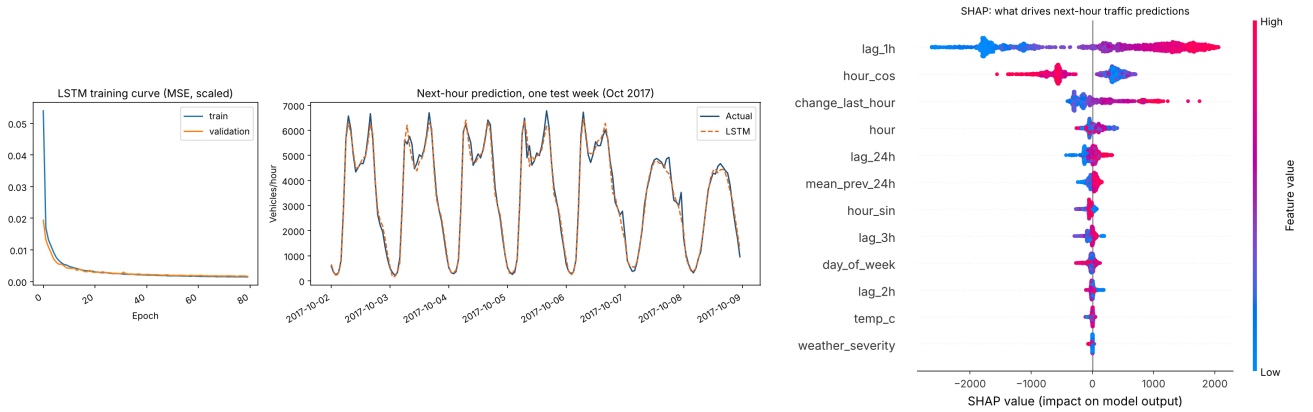


Figure 4. LSTM training curve and one test week (left); SHAP summary for the comparable tree model (right).

5.4 Task 4: Advanced technique, MLflow

Question	Answer
Why MLflow was selected	The project compares many models (6 supervised, 2 unsupervised, 2 deep-learning runs) and needs a deployment path. MLflow gives one place to record what was tried and one registry to decide what is served, and it connects directly to the MLOps tasks. Quantisation or a GAN would add complexity without solving a problem this project has.
How it was implemented	A local SQLite tracking store (mlflow/mlflow.db) with four experiments. Each run logs parameters, metrics, tags and the model with its input signature. Every supervised model is registered as a new version in the Model Registry; the best version receives the alias "champion" and is exported for the API. Monitoring results are logged as a run too. A summary is exported to MLFLOW_SUMMARY.md.
What value it adds	Reproducibility (every result traceable to parameters and data split), fair comparison (all runs share one metric table), governance (an auditable record of which version was promoted and why) and a clean hand-off from experimentation to deployment.
Limitations	The store is local and single-user, with no access control or remote artifact storage. Artifact paths are absolute, so they must be regenerated after cloning. MLflow records what was done but does not judge data quality or fairness, and registry promotion here is rule-based without a human approval step.

5.5 Task 5: Recommendation system

The dataset covers one corridor, so the system recommends **when** to travel. For a requested day type or named day, weather, temperature, month, time window and journey length, it (1) predicts volume for every candidate hour with the champion regressor under exactly those conditions, (2) looks up the share of comparable past hours that were congested, (3) adds the proxy-risk probability, (4) scores every possible window and (5) returns the best window, up to two non-overlapping alternatives that were not historically congested, the window to avoid and a plain-language message. Examples from the system:

- *"For a weekday journey, consider travelling between 10:00 and 11:00, when traffic is expected to be around 4,631 vehicles per hour and 0% of comparable past hours were congested. Good alternatives: 18:00 and 19:00; 13:00 and 14:00. Avoid 16:00 and 17:00: traffic is expected to reach about 6,407 vehicles per hour and 94% of comparable past hours were congested, so your recommended window carries about 28% less traffic."*
- *"For a Friday journey in snow, consider travelling between 20:00 and 21:00, when traffic is expected to be around 2,661 vehicles per hour and 0% of comparable past hours were congested. Good alternatives: 18:00 and 19:00. Avoid 16:00 and 17:00: traffic is expected to reach about 6,014 vehicles per hour and 90% of comparable past hours were congested, so your recommended window carries about 56% less traffic. Snow reduces visibility or grip, so allow extra time and stay clear of the busiest hours, where the proxy risk is highest."*

The recommender is available as a Python class, a command-line tool and an API endpoint. Invalid requests (unknown day or weather, impossible hours) are rejected with a clear message.

5.6 Task 6: MLOps and deployment simulation

6.1 Model versioning. MODEL_VERSIONS.md documents all six registered versions with their metrics, training time and size (the tables in section 5.1). **6.2 Experiment tracking.** MLflow holds 4 experiments and 11 runs with parameters, metrics and model versions. **6.3 Deployment.** A FastAPI service loads the champion models and exposes /health, /predict/traffic, /predict/risk and /recommend, with typed request validation and automatic interactive documentation. Example: a clear Tuesday at 08:00 returns 5,800 vehicles ("Severe"); a foggy January morning at 07:00 returns proxy risk 0.99 with a note that it is not an accident prediction. Nine test calls (six valid, three invalid) all behave as expected.

6.4 Monitoring and 6.5 alerting. Each month of 2018 was treated as a production batch. Prediction-error drift compares monthly MAE with the same month of 2017 (ALERT above +25%). Feature drift uses the Population

Stability Index for temperature, cloud cover, weather type and predictions against the 2016-2017 normal for that month (ALERT above 0.25). The reference deliberately excludes 2012-2015, because monitoring revealed that the recording scheme for cloud cover and weather labels changed around 2015. Result: 4 of 9 months PASS; 4 raise a feature-drift alert graded "watch" (for example February 2018 was 6.5 °C colder than normal, yet accuracy held); and 1 month, April 2018, raises an error-drift alert graded "investigate" (MAE +56%). The cause was a real snowstorm on 14-15 April, when traffic ran 54% below forecast (62% on the 14th). Two simulated incidents confirm the alerts fire: a sensor under-counting by 40% triggers error drift (MAE x5.8) and a 10 °C temperature shift triggers feature drift (PSI 2.3). The overall status is reported as PASS / Normal or ALERT / Requires investigation in a dashboard, a markdown report and a JSON file, and each alert is logged at WARNING level.

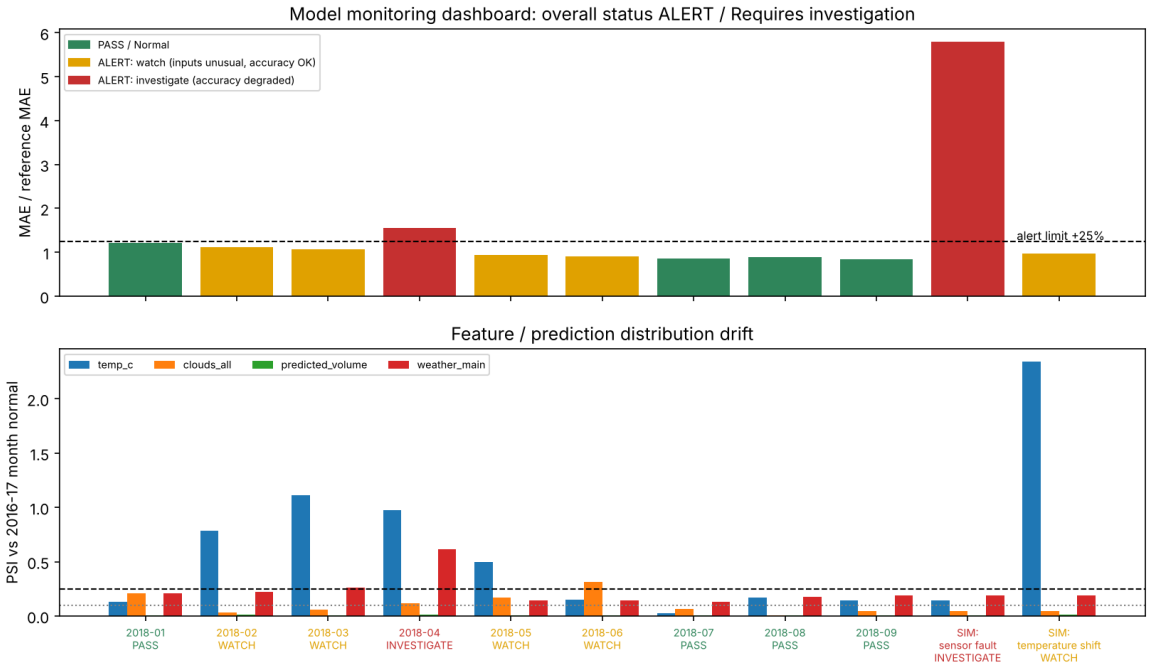


Figure 5. Monitoring dashboard: error drift (top) and feature drift (bottom) by 2018 month, plus two simulated incidents.

5.7 Task 7: Responsible and sustainable AI

The full analysis is in *responsible_ai_report.pdf*. In brief: **coverage** is limited to one westbound sensor with a 10-month gap, 29% of hours unusable for sequence modelling and a change of weather-recording scheme. The **proxy label** is not accident data, can never flag quiet night hours and yields inflated scores because it is built from model inputs. **Errors are uneven**: traffic MAE is 353 in snow and storms against 225 in clear weather, and 493 on holidays against 204 on weekdays, so the system is least reliable when guidance matters most. **Governance**: validation against real incident data, a named human approver, segment-level monitoring and staged release are needed before real-world use. **Sustainability**: the whole Part 3 pipeline runs in about 2.1 minutes on one CPU core; the smaller of two equally accurate models was chosen; and the LSTM cost 15 times the compute of the tree model without beating it.

6. How the three parts fit together

Finding or artefact	Part 1	Part 2	Part 3
Data-quality issues	Discovered with SQL and Power Query	Fixed by logged cleaning steps	Clean table used for all models; sensor faults excluded
Peak hours 07-08 and 15-17	Seen in SQL and dashboard	is_peak_hour feature; heatmap	Top association rules; recommender's "avoid" windows
Weather matters little	r = 0.13; adjusted odds ratio 1.006	Fog: low volume but high congestion share	Low SHAP values for weather; bad-weather cluster less congested
Congestion definition	Fixed threshold 5,500	Quartile categories + is_congested	Proxy label built on quartiles; 5,500 used by recommender
Operational use	Dashboard slicers	CLI application	API, recommender, monitoring

7. Limitations and next steps

- **Single sensor and direction**, with gaps and a change of recording scheme; results may not generalise.
- **Proxy label**: obtaining real crash or incident records is the most valuable next step, so that the classification task can be validated or replaced.

- **Weaker performance in rare conditions** (severe weather, holidays): collect more examples, add event and road-works data, and report prediction intervals.
- **Deployment is a local mock-up**: a real service needs authentication, a live data feed, automated retraining triggered by monitoring, and a human approval step in the registry.

8. Conclusion

The project delivers a coherent pipeline from raw data to an operational AI prototype. Its central finding is consistent across SQL, statistics, visualisation, clustering, rule mining and model explanations: congestion on this corridor is driven by the weekday commuting cycle, and it is predictable enough to give travellers and planners useful advance guidance. The mobility team should focus interventions on the 07:00-08:00 and 15:00-17:00 weekday windows, use the recommender to encourage off-peak travel, keep monitoring switched on, and treat the risk output strictly as a proxy until real incident data are available.